

Remote Shutdown — Synthèse d'architecture

tgayet@tixeo.com

15 avril 2026

Contexte

Éteindre à distance un ordinateur depuis un poste **Linux Debian/Ubuntu**. Les cibles sont en réseau et tournent sous **Windows 10/11**, **Linux (Debian/Ubuntu/Fedora)** ou **macOS**. Un logiciel/agent peut être installé sur la cible.

Options évaluées

Option 1 — SSH (recommandée)

OpenSSH Server est disponible **nativement** sur les trois familles d'OS :

- **Linux** : `apt install openssh-server / dnf install openssh-server`
- **Windows 10/11** : `Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0`
- **macOS** : « Remote Login » dans Préférences Système (OpenSSH livré d'origine)

Schéma :

```
[Debian/Ubuntu] --SSH (ED25519)--> [Cible Win / Linux / macOS]
                                |-> shutdown local
```

Commandes d'arrêt :

- Linux / macOS : `sudo shutdown -h now`
- Windows : `shutdown /s /t 0`

Durcissement via `~/.ssh/authorized_keys` côté cible :

```
restrict,command="shutdown -h now",from="10.0.0.5" ssh-ed25519 AAAA...
```

La clé ne peut **rien faire d'autre** que déclencher l'arrêt, et seulement depuis l'IP autorisée.

Option 2 — Agent Go + HTTPS/mTLS

Un binaire léger (Go, cross-compilable en un seul source pour Windows, Linux, macOS Intel et Apple Silicon) expose `POST /shutdown` protégé par mTLS + JWT signé avec nonce anti-rejeu. S'installe comme service : `systemd` (Linux), service Windows, `launchd` (macOS).

Pertinent si : API REST requise, audit centralisé signé, refus politique d'exposer SSH.

Option 3 — Broker MQTT/AMQP

Centralisation par publish/subscribe, utile uniquement au-delà d'une dizaine de postes pilotés simultanément. Sinon : sur-dimensionné.

Options écartées

- **WinRM / PowerShell Remoting** : Windows-only.
- **Apple Remote Desktop** : macOS-only.
- **Wake-on-LAN** : sert à allumer, pas à éteindre.
- **Outils RMM (MeshCentral, Ansible Tower...)** : disproportionnés pour un simple **shutdown**.

Comparatif

Critère	SSH	Agent mTLS	MQTT
Couverture Win/Linux/mac	natif	via Go	via Go
Effort de mise en oeuvre	faible	moyen	élevé
Sécurité par défaut	_____	_____	_____
Maintenance	quasi nul	code custom	broker
Audit / compliance	logs SSH	sur-mesure	broker logs

Recommandation

Adopter **SSH**, pour trois raisons :

1. **Zéro développement** : les trois OS cibles embarquent OpenSSH — ce n'est pas un logiciel propriétaire mais un service standard auditable.
2. **Sécurité éprouvée** : clé ED25519 dédiée + restrictions `command= / from=` + pare-feu = surface d'attaque minimale. En cas de compromission de la clé, elle ne permet **que** d'éteindre la machine.
3. **Portabilité** : un même wrapper CLI côté Debian pilote les trois OS ; seule la commande distante change.

Basculer sur **l'agent Go + mTLS** uniquement si une exigence impose : API REST, audit signé centralisé, interdiction d'exposer SSH.

Architecture SSH proposée

Poste contrôleur (Debian/Ubuntu)

- Clé dédiée : `ssh-keygen -t ed25519 -f ~/.ssh/remote_shutdown -C "remote-shutdown"`
- Inventaire des cibles dans `~/.ssh/config` (host, user, IdentityFile)
- Script `remote-shutdown <alias>` encapsulant l'appel SSH et journalisant localement (`~/.local/state/remote-shutdown.log`)

Postes cibles (provisionnement unique)

- Activer le serveur SSH (commandes OS ci-dessus)
- Créer un utilisateur local non-privilégié `ctrlshut`
- Autoriser uniquement la commande d'arrêt :
 - Linux / macOS : `sudoers ctrlshut ALL=(ALL) NOPASSWD: /sbin/shutdown -h now`
 - Windows : accorder `SeShutdownPrivilege` au compte
- Déposer la clé publique avec restrictions dans `authorized_keys`
- Pare-feu : port 22 ouvert **uniquement** depuis l'IP du contrôleur

Prochaines étapes

Décisions à arrêter avant implémentation :

1. Nombre de postes cibles et répartition par OS
2. CLI simple ou TUI (fzf / gum)
3. macOS sous FileVault ? (impact au redémarrage, pas à l'arrêt)
4. Exigence de log centralisé (syslog, journald remote, SIEM)